COMP 018: Database Administration - Activity 4 Progress Report

Access Control, Database Users, Roles, Privileges, and Security Governance

Project Identification & Metadata

Course	COMP 018 Database Administration
Evaluating Group	Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	https://github.com/Nycri/brewpos
Target Project Monitored	Restaurant Online Ordering & Table Reservation System (Mesa / Binary Sisters)
Target Project Group	Group 10: Sandoval, Cellyn; Callos, Kathlen Mae; Romingquit, Arika; Tambio, Jenny Mae; Tungol, Jaimie; Gordonas, John Benedick
Target Project Repository	https://github.com/kathlenmaecallos11-arch/Restaurant-Online Ordering-Table-Reservation-System
Primary Database	grp_binarysisters_projectdb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	grp_binarysisters_projectdb_restore_test

ACTIVITY 4: Database Users, Roles, Privileges, and Access Control

1.1 Access-Control Plan & Architecture

In default database configurations, web backend applications frequently connect using an administrative database superuser account. Operating an interactive online restaurant ordering and table reservation platform under an administrative superuser account severely violates the Principle of Least Privilege. If an unauthenticated user were to exploit an application vulnerability or public API route, they would possess unrestricted data definition and

manipulation capabilities across the database cluster, threatening customer accounts, dining reservations, dish pricing, and financial revenue audits.

To establish secure operations for Group 10's Restaurant Online Ordering system, a comprehensive role-based access control architecture was developed. This architecture decouples transactional order processing, operational restaurant management, and compliance auditing from administrative database maintenance. The design implements non-login group roles to serve as structural privilege templates alongside dedicated login accounts assigned to their matching group roles.

Under this security plan, the application runtime account holds read, insert, and update capabilities across active transactional tables, while its row deletion rights are strictly confined to pending order line items during customer cart assembly. Row deletion remains strictly prohibited on customer profile accounts, staff rosters, menu dishes, and completed orders. Read-only reporting and management accounts are granted query access across relational views to review dining metrics and sales without direct table modification rights, and are strictly denied access to sensitive user credential tables to shield employee and customer password hashes. Dedicated maintenance and backup operator roles are established with narrow scopes and explicit connection throttling limits.

1.2 Access-Control Role Definitions and Privilege Structure

The application runtime user operates under a dedicated group role assigned to handle dining reservations, menu catalog lookups, customer orders, and cart line items. Its permissions maintain full selection, insertion, and update capabilities on operational tables while confining deletion strictly to pending order items, protecting master dishes, customer identities, and billing records from deletion.

The read-only reporting account operates through an auditing group role to review dining revenue, reservation timelines, and order summaries via pre-compiled relational views (v_active_menu, v_order_details, v_today_reservations). It holds read access on operational reporting structures while omitting read permissions on raw customer and staff credential tables.

The backup operator account is assigned dedicated read permissions across tables and sequences to support logical dump generation without write or schema alteration permissions.

Database administration tasks remain strictly reserved for the database administrator superuser account.

1.3 Access-Control Configuration Summary

The configuration process established five non-login group roles representing runtime, read-only audit, maintenance, backup, and DBA profiles, followed by the creation of matching login accounts with dedicated authentication credentials and connection throttling limits (CONNECTION LIMIT 20 for application runtime). Access privileges were revoked from public roles across the database and schema to enforce explicit access assignments. Connection

privileges and schema usage rights were assigned to the designated group roles, explicit table grants were applied according to operational requirements, and default schema privileges were established to preserve access rules across future database objects.

1.4 Verification Results: Runtime Account Testing

Verification testing demonstrated that the runtime account successfully queries active menu items, places customer orders while advancing sequence counters, updates table reservation statuses, and queries operational dining views. When row deletion operations were attempted on customer accounts, staff rosters, or menu dishes, the database engine denied execution under the narrow deletion policy. Data definition commands, such as dropping notification or menu tables, were rejected due to missing object ownership permissions.

1.5 Verification Results: Read-Only Auditor Account Testing

Inspection testing confirmed that the read-only auditing account successfully executes multi-table queries across dining views, including itemized order histories and daily reservation lists. When query attempts were made on customer and staff credential tables containing authentication hashes, access was denied by the database security layer. Attempted modification operations, including inserting new orders, updating reservation schedules, or deleting menu dishes, were uniformly blocked by the engine.

1.6 Verification Results: Cross-Database & Administrative Boundary Testing

Administrative isolation testing showed that database creation commands attempted under application accounts were denied due to missing administrative creation privileges. Connection attempts to unrelated databases on the server instance failed because public connection permissions were revoked, confining access strictly to authorized database targets.

1.7 Issues, Fixes, and Feedback to Owners

During initial deployment, references to a missing password initialization script were identified in setup documentation. We resolved this by establishing explicit role passwords with enforced connection throttling limits directly within PostgreSQL, verifying all eight tables, four reporting views, and reservation triggers.
Feedback provided to Group 10 commended the exemplary relational schema design, extensive utilization of normalized database views, and automated reservation state triggers.
Recommendations advised implementing an explicit check constraint on reservation party sizes

(guest_count > 0) to prevent zero-capacity bookings, alongside adding a unique constraint on active reservation time slots per dining table.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 4)

Column Header	Value
Project	Group 10 - Restaurant Online Ordering & Table Reservation System (Binary Sisters)
Repository cloned	https://github.com/kathlenmaecallos11-arch/Restaurant-Online Ordering-Table-Reservation-System, Branch: main
Database setup result	Success (Verified PostgreSQL 18.6 on port 5432, schema built with 8 tables, 4 views, triggers, constraints, populated sample data, configured least-privilege roles).
Access-control folder link	https://github.com/Nycri/brewpos/tree/main/access-control
Access-control status	Completed
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity4_Progress_Report_Group10.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.